

CS246E—Deliverables, Project (Fall 2025)

B. Lushman

Due Date 1: Friday, November 21, 5pm
Due Date 2: Friday, December 12, 11:59pm

Note that since the project is due during the exam period, it is not eligible for a 48-hour absence declaration. Failure to submit the project will result in a grade of INC in the course, subject to the usual rules about eligibility for INCs.

Your project will be graded based on correctness and completeness (60%), documentation (20%), and design (20%). Grading will proceed in two phases, outlined below:

If you declared a 48-hour absence on the midterm, then the weight of the midterm transfers to the project, but you have to do a larger project. The way this will work is that your project grade will start with a deficit of 10%. This 10% can be earned back, but you must then submit a proposal for what you will add to your project in order to earn that 10%. You will still be eligible for bonus credit.

Correctness and Completeness

The correctness and completeness component of your project will be assessed via a demo in front of the ISA or a TA. You will need to sign up for a 20-minute demo slot. The demo will be done either in-person or via screen sharing over MS Teams. If neither of these are possible for you (e.g., due to poor internet service), it is possible to have the TA mark your project without you present. In that case, you must submit a “guide” to your project, indicating the key features of your project, and any ways in which you deviated from the specification. This is to make sure the TA doesn’t miss anything as a result of you not being present. If possible, however, we recommend that you participate in your demo, as it will lead to faster marking, and likely fewer errors.

After you have demonstrated the core requirements, you may demonstrate any additional features of the project that you may have implemented, for extra credit (if your demo is not live, outline these features in your guide document). The TA will assess the worth of your additional features after your demo is over.

Documentation and Design

The design of your project is an important part of your overall assessment. For top grades, it is not enough that your program simply work; it must demonstrate a solid object-oriented design. Thus, you will need to spend considerable time planning out your classes and their interactions, aiming to minimize coupling while maximizing cohesion.

Your documentation and design will be assessed via written documents that you will submit to Marmoset as pdfs.

Extra Credit Features

You have the opportunity to earn up to 10% extra credit for enhancements to the core project. Keep the following rules and guidelines in mind when planning enhancements:

- It is far more important to have your core project working well, than to have a poorly done project with extras. Remember, the enhancements are only worth 10%. Your documentation and design are also more valuable than your enhancements, so please take time to do these well.
- Your program must be able to run the core project, without enhancements, as well as with them. You are not allowed to submit two programs for grading, one with enhancements and one without. You are also not allowed to recompile your program with different compiler flags to produce versions with and without enhancements. You may select or deselect your enhancements via flag arguments on the command line. (E.g. `./project -enhanced`, or something similar.) Even better would be if you could turn your enhancements on and off dynamically, as the program is running.
- Markers will not assign values to individual bonus features during the demo. You may show what you have done, and the TA will take note, but values will be decided once all demos are completed.
- One enhancement is offered as a challenge: complete the entire project, without leaks, and without explicitly managing your own memory. In other words, handle all memory management via vectors and smart pointers. If you do this, there should be no delete statements in your program at all. Successful completion of this challenge will earn you 2 of the available 10 bonus marks. If your program leaks, these marks cannot be earned. Also note that these 2 marks are not reserved for this feature; it is theoretically possible to get all 10 bonus marks without implementing this challenge. (Note that this challenge is usually offered to CS 246 as well, for 4 marks. The thinking here is that you have been aware of RAII for much longer than CS 246, so this offer is more of an expectation, and not as much of a challenge.)

On Due Date 1

On Due Date 1, you must submit a plan of attack and a milestone implementation. The plan of attack should include a UML that describes how you anticipate your system to be structured (even if you complete the project by due date 1, the UML you submit must be one that you built prior to starting the project).

Your UML should show the classes that make up your project and the relationships among them. You only need to show public methods (i.e, you can leave out private fields and protected/private methods, unless you need to show them to illustrate a point, e.g., a design pattern). Do not show the big 5 operations, or any other constructors, accessors, or mutators. You will not be graded on the degree to which you adhere to this model, but you will be asked to account for any differences that arise between this model and your final submission. File: `uml.pdf`.

In addition, your plan of attack must include a breakdown of the project, indicating what you plan to do first, what will come next, and so on. Include estimated completion dates. You should try to stick to your plan, but you will not be graded by the degree to which you stick to it. Your initial plan should be realistic, and you will be expected to explain why you had to deviate from your plan (if you did). Your plan should be no more than 3 pages long. File: `plan.pdf`

The milestone implementation must be a very basic start to your project that demonstrates that you have begun working on it. For the project options that are not self-proposed, your milestone implementation must at a minimum be able to enter ncurses mode, display something on the screen (text in the case of vm, game objects in the case of AGE and Brad is You) and take some action in response to keystrokes. For a self-proposed project, you must include a description of a similar minimally-working start to the project in your proposal.

In either case, you should submit whatever code you have at the time of Due Date 1, to Marmoset, together with a Makefile, such that issuing the command `make` builds the milestone implementation of your project.

On Due Date 2

On Due Date 2, you must submit all of your code to Marmoset, together with a Makefile, such that issuing the command `make` builds your project.

In addition, you must submit a final design document. It must outline the final, actual design of your project, and how it differed from your design on Due Date 1 (if it did). You should include an updated UML, reflecting the actual structure of your project. Do this even if it is the same as the original UML.

Your document should provide an overview of all aspects of your project, including how, at a high level, they were implemented. If you made use of design patterns, clearly indicate where.

Your system should employ good object-oriented design techniques, as presented in class.

You should not expect the TA to read through all of your code. Therefore, your design document should stand alone – the TA should not need to have your code open in order to understand your document. However, if you wish to highlight certain aspects of your design, you should indicate clearly where they can be found in your code, if the TA wants to have a look.

Finally, answer the following question:

1. What would you have done differently if you had the chance to start over?

Length and Format of Document

We expect that your final document will be 4–9 pages long, where each page has a word density similar to what you see in this guideline document.

Organize your document into sections, using the following headings:

- Introduction (if needed)
- Overview (describe the overall structure of your project)
- Updated UML
- Design (describe the specific techniques you used to solve the various design challenges in the project)
- Extra Credit Features (what you did, why they were challenging, how you solved them—if necessary)
- Final Question (the question in this document).
- Conclusion (if needed)

The document you submit will be used for both your documentation mark and your design mark. Your design will be assessed according to how you solved the problem, as described in your document, and in your Due Date 1 UML. Your documentation will be graded according to how well you described your project, how well you described your design (i.e., is it clear and well-communicated?), and your Due Date 1 document. **Together, these account for 40% of your grade, so these documents are very important.** The Due Date 2 document is worth much more than the Due Date 1 document.

Late Policy

We **strongly** discourage you from making last-minute changes to your code. The risk of your code not compiling, or otherwise not working, is too high. We recommend writing no new code in the last six hours before the code is due. Save those hours for preparing for your demo, and for emergency bug fixes.

Late assignments will be dealt with harshly. **Assignments submitted late will be deducted 1/12 of the total mark for every hour late, rounding up.** Therefore, an assignment submitted more than 11 hours late earns 0 marks. The lateness of your project will be based on your *last* submission to Marmoset, whether it compiles or not. So, before you contemplate adding a last minute feature, be sure it is worth the risk of not getting it done on time.